

Ablation study of shared preprocessing vs backbone-specific preprocessing

Bashayer Marghalani

June 2026

1 Preprocessing and Feature Extraction

We followed the outlines of previous research on brain tumour classification using MRI, including preprocessing, feature extraction, classification, and evaluation.

1.1 Preprocessing

Basically, image Augmentation is applied to the dataset, see Fig. 1. Let $x_i \in \mathcal{D}$, where $\mathcal{D}$. The implementation applies four image views of x_i : the original pre-processed image x_i , its horizontal flip, and rotations by $+15^\circ$ and -15° . If $\tau_m(\cdot)$, $m \in \{1, 2, 3, 4\}$, denotes these four transformations, then the transformed views are:

$$x_i^{(m)} = \tau_m(x_i), \quad m = 1, \dots, 4. \quad (1)$$

Subsequently, each view was resized to 224×224 , converted to a tensor, and normalised using ImageNet statistics before the extraction of features.

The skull stripping was not performed here; The MRI images are used with their full view to challenge the classifier.

1.2 Feature Extraction

The CNN features have been chosen after investigating the performance of many pretrained networks on the datasets used, and extracted features from AlexNet, VGGNet19, VGGNet16, and EfficientNet B0, which show good overall accuracy.

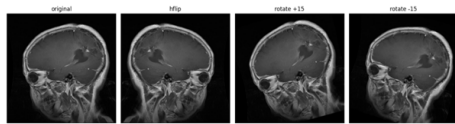


Figure 1: Augmentation has been done to one example image

The model uses four frozen pretrained CNN feature extractors: $\phi^{(b)}$, $b \in \{1, 2, 3, 4\}$ Corresponding to AlexNet-fc7, VGG16-fc7, VGG19-fc7, and EfficientNet-B0 pooled features, respectively. For each transformed image $x_i^{(m)}$, the backbone-specific feature vectors are

$$z_i^{(m,b)} = \phi^{(b)} \left(x_i^{(m)} \right). \quad (2)$$

The code then performs feature-level fusion by concatenation:

$$z_i^{(m)} = z_i^{(m,1)} \oplus z_i^{(m,2)} \oplus z_i^{(m,3)} \oplus z_i^{(m,4)}. \quad (3)$$

where $\oplus$ denotes vector concatenation. Finally, the four fused representations are averaged to obtain a single feature vector per original image:

$$f_i = \frac{1}{4} \sum_{m=1}^4 z_i^{(m)}. \quad (4)$$

Thus, the fused feature matrix is denoted by

$$F = f_1^\top f_2^\top \dots f_N^\top. \quad (5)$$

In the backbone-specific preprocessing ablation, the shared preprocessing operator $\pi(\cdot)$ is replaced by backbone-dependent operators $\pi^{(b)}(\cdot)$, such that

$$x_i^{(m,b)} = \pi^{(b)} \left(A_m(x_i) \right). \quad (6)$$

$$z_i^{(m,b)} = \phi^{(b)} \left(x_i^{(m,b)} \right). \quad (7)$$

The fusion and averaging stages remain unchanged, thereby isolating the effect of the preprocessing regime alone.

1.2.1 The backbone-specific preprocessing

According to the specific ablation code used here, the default built-in transformations are extracted for each individual neural network using PyTorch’s weight mechanisms. The code defines separate transforms for each backbone:

- `T_vgg16 = vgg16_w.transforms()`
- `T_vgg19 = vgg19_w.transforms()`
- `T_eff = eff_w.transforms()`
- `T_alex = alex_w.transforms()`

That means, when an MRI or CT scan of a tumour is passed through these specific transforms, the image is altered in different ways before it even reaches the neural network. For example,

- VGG16/19 might resize the image to 256x256, take a centre crop of 224x224 using bilinear interpolation, and normalise using ImageNet standards.
- EfficientNet might use a completely different interpolation method (like bicubic), a different crop size (e.g., 256x256 or 288x288 depending on the B-variant), and different normalisation statistics.

This leads to misaligned features z_i , because `T_vgg16`, `T_eff`, and the other backbone-specific preprocessing pipelines apply different cropping and scaling rules. As a result, the spatial layout of the tumour in the resulting tensor may change across backbones.

In Equation 2, $x_i^{(m,b)}$ denotes The image was transformed by a backbone-specific pipeline. Therefore, the inputs may be spatially shifted or scaled differently, and the resulting one-dimensional feature vectors z_i may represent slightly different physical regions of the original scan. For example, the feature vector extracted by VGG16 may strongly encode the centre of the tumour, while the feature vector extracted by EfficientNet may focus on different texture patterns due to differences in scaling and interpolation.

According to Equation 3, the specific-preprocessing architecture concatenates these features and then averages them across views.

This process may combine feature representations that are not spatially consistent across backbones. For example, the model may combine a feature representing the tumour boundary from VGG with a feature representing background tissue from EfficientNet. Because the specific preprocessing pipelines shift their spatial focus, this may weaken the medical context required to distinguish a pituitary tumour from a meningioma, or a healthy brain from a glioma.

Shared preprocessing avoids this issue by applying a single unified transform, such as standard `transforms.Compose` resizing to 224×224 with standard normalisation, to all images before they are passed to the backbones AlexNet, VGG16, VGG19, and EfficientNet. Therefore, the same tensor is provided to all four backbones. In this case, each convolutional neural network observes the same pixels. For example, if there is an artefact or tumour boundary at pixel location (100, 100), every backbone receives it at the same location.

The shared preprocessing approach showed more robust and stable performance than the backbone-specific preprocessing approach, where the performance decreased. The reasons are Input consistency and feature alignment, shared preprocessing reduced variance,

Eventually, in terms of shared preprocessing vs specific preprocessing For the backbone features, shared preprocessing shows robust, stable performance, whereas specific preprocessing has fallen off. The features which have been extracted from each backbone are covering and representing different areas and aspects of the brain MRI image as it shown in Figure 2.

1.2.2 Principal Component Analysis (PCA)

PCA was subsequently fitted on the training split of these averaged fused features and applied unchanged to the validation and test sets.

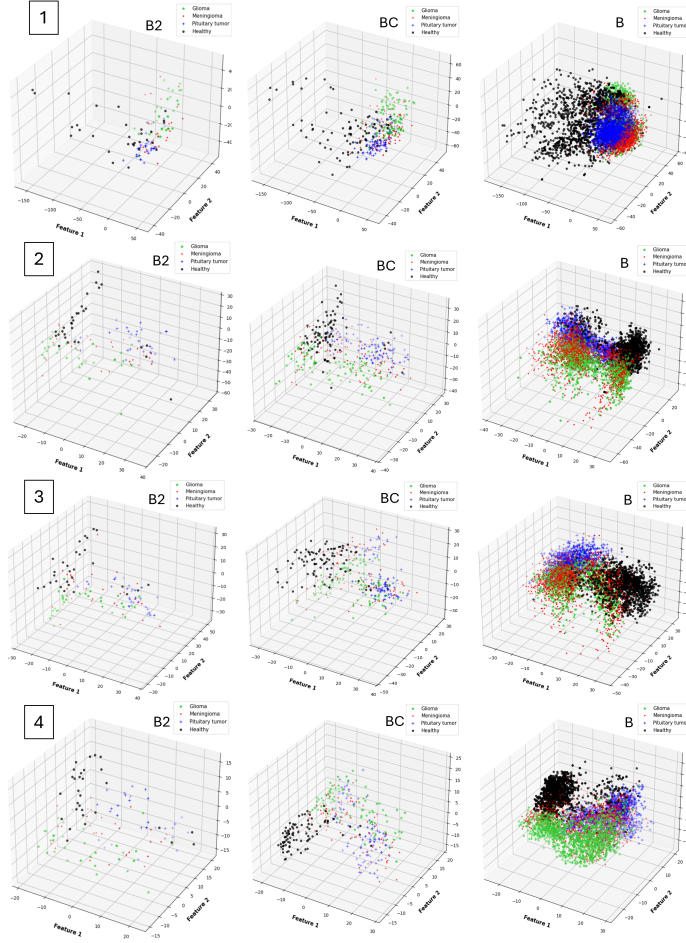


Figure 2: 3D scatter plot of 1-AlexNet, 2-VGG16, 3-VGG19, and 4- EfficientNet extracted features of 4 classes (dataset B2, BC and B)